# 4) Password policies


- PAM module
  - Why PAM Exists
  - Example
  - PAM Configuration Files
  - PAM Modules
- Password expiration
- Password complexity
- Enforcing user and root to comply for password rules by PAM
- Sudo configuration
  - Changing the number of tries for a user to use sudo.
  - Changing the Custom message appears when fail password entered for sudo.
  - Each action using sudo has to be archived and saved in a log file.
  - Enable TTY mode
  - Restrict the paths used by sudo


| Requirement | File |
|---|---|
| Password expiration | `/etc/login.defs` |
| Password complexity | `/etc/security/pwquality.conf` |
| Password history | `/etc/pam.d/system-auth` |
| Authentication system | PAM |
| `Sudoers` configuration | /etc/sudoers |

---

## PAM module

- PAM i.e **Pluggable Authentication Modules** is a **modular system** that separates **authentication logic** from applications.
- It is a Linux framework that manages authentication and password policies through configurable modules used by applications like login, SSH, and passwd.
- This means programs like:
  - `login`
  - `ssh`
  - `su`

- `passwd`
  do **not implement authentication themselves**.
  Instead, they ask **PAM** to handle authentication.

## Why PAM Exists

Before PAM, every program had to implement authentication separately.

PAM allows:

- centralized authentication rules
- flexible security policies
- easy modification without changing applications

## Example

When you run:
`passwd`
The process is:

```
passwd command
      ↓
PAM modules
      ↓
password policy check
      ↓
password accepted or rejected
```

So PAM enforces rules like:

- minimum password length
- uppercase letters
- password history
- expiration policies

## PAM Configuration Files

Main directory:
`/etc/pam.d/`

Example files:

| File | Purpose |
|---|---|
| `/etc/pam.d/sshd` | SSH authentication rules |

| File | Purpose |
|---|---|
| `/etc/pam.d/login` | Console login rules |
| `/etc/pam.d/system-auth` | Common system authentication |
| `/etc/pam.d/passwd` | Password change rules |

## PAM Modules

PAM works through **modules**, for example:

| Module | Purpose |
|---|---|
| `pam_pwquality.so` | password strength rules |
| `pam_unix.so` | standard UNIX authentication |
| `pam_pwhistory.so` | password history |
| `pam_faillock.so` | account lock after failed logins |

---

# Password expiration

- `/etc/login.defs`

```
# Password aging controls:
#
#       PASS_MAX_DAYS   Maximum number of days a password may be used.
#       PASS_MIN_DAYS   Minimum number of days allowed between password changes.
#       PASS_MIN_LEN    Minimum acceptable password length.
#       PASS_WARN_AGE   Number of days warning given before a password expires.
#
PASS_MAX_DAYS   30
PASS_MIN_DAYS   2
PASS_MIN_LEN    8
PASS_WARN_AGE   7
```

- For acceptable password length and password complexity enforces, we will use PAM module.

---

# Password complexity

- `/etc/security/pwquality.conf`

```
#
# Minimum acceptable size for the new password (plus one if
# credits are not disabled which is the default). (See pam_cracklib manual.)
# Cannot be set to lower value than 6.
minlen = 10
#
# The maximum credit for having digits in the new password. If less than 0
# it is the minimum number of digits in the new password.
dcredit = -1
#
# The maximum credit for having uppercase characters in the new password.
# If less than 0 it is the minimum number of uppercase characters in the new
# password.
ucredit = -1
#
# The maximum credit for having lowercase characters in the new password.
# If less than 0 it is the minimum number of lowercase characters in the new
# password.
lcredit = -1
#
# The maximum credit for having other characters in the new password.
# If less than 0 it is the minimum number of other characters in the new
# password.
# ocredit = 0
#
# The minimum number of required classes of characters for the new
# password (digits, uppercase, lowercase, others).
# minclass = 0
#
# The maximum number of allowed consecutive same characters in the new password
# The check is disabled if the value is 0.
maxrepeat = 3
```

---

## Enforcing user and root to comply for password rules by PAM

- If you try as `root` to change the password for any account (including root), it will just warning you that you are not complying with the rules specified by PAM, and not enforcing you as a `root`.

```
[root@aqoraan42 ahmadquraan]# passwd ahmadquraan
New password:
BAD PASSWORD: The password contains less than 1 uppercase letters
Retype new password: _
```

- To solve this we need to enforce that in `/etc/pam.d/system-auth`.

```
auth        required                    pam_env.so
auth        required                    pam_faildelay.so delay=2000000
auth        sufficient                  pam_unix.so nullok
auth        required                    pam_deny.so

account     required                    pam_unix.so

password    requisite                   pam_pwquality.so try_first_pass local_users_only retry=3
password    sufficient                  pam_unix.so yescrypt shadow nullok use_authtok
password    required                    pam_deny.so

session     optional                    pam_keyinit.so revoke
session     required                    pam_limits.so
-session    optional                    pam_systemd.so
session     [success=1 default=ignore]  pam_succeed_if.so service in crond quiet use_uid
session     required                    pam_unix.so
```

- Add on `password requisite pam_pwquality.so -->`:
  - `enforce_for_root` : To force the password quality checks on root too.
  - `local_users_only` : The password quality checks apply **only to local users stored on the system** (in `/etc/passwd`).
  - `retry=3` : How many tries do you have before exit.

---

## Sudo configuration

- Sudo configuration file is in : `/etc/sudoers`
- Shortcut for it using `sudo visudo`

### Changing the number of tries for a user to use sudo.

```
# In /etc/suderos add this line:
Defaults passwd_tries=N
```

> **The approach used to change the tries for sudo is differ than the one used for pwquality. Both for different things.**

### Changing the Custom message appears when fail password entered for sudo.

```
Defaults badpass_message="Wrong pass"
```

### Each action using sudo has to be archived and saved in a log file.

```
Defaults log_input
Defaults log_output
Defaults iolog_dir="/var/log/sudo" # Example file
```

| Setting | Meaning |
|---|---|
| `log_input` | logs user keyboard input |
| `log_output` | logs command output |
| `iolog_dir` | directory where logs are stored |

- In `/var/log/sudo` a list of directories and files will be made each time a **user use `sudo`** :

```
/var/log/sudo/
├── seq
└── 00
    └── 00
        └── 0A
            ├── log
            ├── log.json
            ├── stdin
            ├── stdout
            ├── stderr
            ├── timing
            ├── ttyin
            └── ttyout
```

| File | Meaning |
|---|---|
| `stdin` | what the user typed |
| `stdout` | command output |
| `stderr` | errors |
| `ttyin` | terminal input |
| `ttyout` | terminal output |
| `timing` | replay timing |
| `log.json` | metadata about the command |

- Now each time a user write something with `sudo`, it gonna be saved in `/var/log/sudo`.
- You can **see the records using** `sudo sudoreplay -d <log directory> <session_id>`
- **List the sessions id** using `sudo sudoreplay -d <log dir> -l`
- Summary:

```
sudo command
    ↓
record session
    ↓
store in /var/log/sudo/sessionID
    ↓
replay with sudoreplay
```

## Enable TTY mode

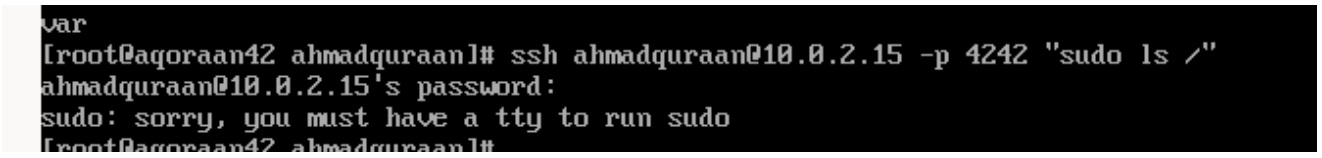
- This mode forces `sudo` to **run only from a real terminal (TTY)**, so `sudo` **will fail if there is no interactive terminal**.
- Example of blocked usage:
  `ssh user@server "sudo ls"`

> It require a valid interactive terminal

- Apply it in `/etc/sudoers` or `sudo visudo`

```
Defaults requiertty
```

- Verify it's working:
  `sudo sudo -V | grep tty`




## Restrict the paths used by sudo

- In `/etc/sudoers` add:

```
Defaults secure_path=...
```